

What is a function? List its advantages
Positional, keyword, default, *args, **kwargs

What is the return statement?

Explain LEBB rule.

What is recursion? Give an example looking

Coding
Functions to return factorial of a number
Function to return largest element in a list
Recursive fibonacci function.

A function is a block of code designed to perform a specific task. It can take inputs, process them, and return an output. Functions help organize code into modular, reusable and manageable components making complex programs easier to read and maintain.

Advantages of using functions:

Code Reusability: Functions enable reuse of the same code multiple times across a program, saving effort and reducing duplication.

Modularity: Breaking a program into smaller functions makes it easier to manage, write, debug and maintain.

Abstraction: Functions hide internal details, allowing users to use

Readability: Isolating functions facilitates debugging without having to fix the whole program.

Testing and Debugging
Testing small code independently improves reliability.

Ques 1) Positional Arguments
Passed in the order. The position matters.

2) Keyword Arguments
You specify the arguments explicitly. Order doesn't matter.

3) Default Arguments
Provide a default value for an argument. Becomes optional.

4) Variable Positional Arguments
Collects extra arguments into a tuple. Useful when the number of arguments is not fixed.

its advantages
alt, *args, **kwargs
statement?

e an example looking

rial of a number
est element in a list
ion.

code designed to
It can take inputs,
an an output
e into modular,
le components
asier to read

ctions:

enable reuse
times across
d reducing

rogram into
s it easier to
nd maintain

internal

them without worrying about how they
work internally

Readability: Isolating easier tasks within
functions facilitates easier updates and
bug fixed without affecting the entire
program.

Testing and Debugging: functions allows
testing smaller parts of the
code independently, enhancing
reliability.

Ques 1) Positional Arguments
Passed in the order they are defined
The position matters

2) Keyword Arguments

You specify the parameter name
explicitly
Order doesn't matter

3) Default Arguments

Provide a default value so the
argument becomes optional.

4) Variable Positional Arguments

collects extra positional arguments into
a tuple

useful when you don't know how many

5. (Variable keyword Arguments)

Collects extra keyword arguments into a dictionary.
Useful for flexible functions.

(like len, print)

Ans

Factorial of

n

is defined as:

$$n! = n \times (n-1)$$

Recursive definition

$$n! = n \times (n-1)!$$

Fibonacci Sequence

Fibonacci

$0, 1$

Recursive definition

$$F(n) = F(n-1) + F(n-2)$$

Ans

L - Local

Name defined inside the current function.

Example: variables created within a function body

E - Enclosing

Name in any outer (but non-global) functions.

Applies in nested functions (inner functions can access variables from outer functions)

G - Global

Name defined at the top level of a module / script or declared global inside a function

B - Built in

Name preassigned in Python

(like len, print, range).

SAns

Factorial of

n

is defined as:

$$n! = n \times (n-1) \times (n-2) \dots \times 1$$

Recursive definition

$$n! = n \times (n-1)! \text{ with base case } 0! = 1$$

Fibonacci Sequence

Fibonacci numbers

0, 1, 1, 2, 3, 5, 8

Recursive definition

$$F(n) = F(n-1) + F(n-2), F(0) = 0, F(1) = 1$$

```
[35]
✓ 0s
def factorial_iterative(n):
    if n < 0:
        return
    result = 1
    for i in range(2,n + 1):
        result *= i
    return result
print(factorial_iterative(5))

120

[38]
✓ 0s
def find_largest(lst):
    if not lst:
        return None
    largest = lst[0]
    for num in lst:
        if num > largest:
            largest = num
    return largest

numbers=[10,25,3,99,42]
print(find_largest(numbers))
```

⌵

Digiicampus

⌵

Welcome To Colab - Colab

⌵

Untitled36.ipynb - Colab

⌵

+

⌵

colab.research.google.com/drive/1yHLOW43b5RV2kVEOHn9iDzUqmjvX1fXJ#scrollTo=aZlrNtC_Zyy-

☆

👤

⋮

CO

Untitled36.ipynb

☆

☁

File

Edit

View

Insert

Runtime

Tools

Help

🗨

⚙

👤 Share

👤

🔍 Commands

+ Code

+ Text

▶ Run all

✓

RAM

Disk

☰

🔍

⏮

⏭

🔑

📁

[35]

✓ 0s

```
for i in range(2,n + 1):
    result *= i
return result
print(factorial_iterative(5))
```

120

[38]

✓ 0s

```
def find_largest(lst):
    if not lst:
        return None
    largest = lst[0]
    for num in lst:
        if num > largest:
            largest = num
    return largest

numbers=[10,25,3,99,42]
print(find_largest(numbers))
```

99

[1]

▶ Start coding or [generate](#) with AI.

↑

↓

✎

🗑

⋮

{} Variables

📄 Terminal

⬡

✓ 8:50 PM

📄 Python 3

```
[44] ✓ 0s def fibonacci(n):  
    """  
    Return the nth Fibonacci number using recursion.  
    Fibonacci sequence:  
    F(0)=0, F(1)=1  
    F(n)=F(n-1)+F(n-2)  
    """  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fibonacci(n-1) + fibonacci(n-2)  
print(fibonacci(6))  
print(fibonacci(10))
```

Output: 8, 55